Python Programming for Finance — Lecture Notes

MSc in Financial Markets and Investments
Skema Business School
Academic Year 2025/26 — Fall 2025
Lecturer: Alexandre Landi

1. Purpose & Learning Objectives

By the end of this project, you will be able to:

- Fetch market and macroeconomic data from FRED programmatically.
- Build a **Streamlit** dashboard with user controls and export.
- Create clear charts: time-series line plots and latest-value bar snapshots.
- Handle data frequency mismatches, missing values, and consistent labeling.

2. Final Deliverable

A working **Streamlit application** (`streamlit run app.py`) that:

- Loads data from FRED for:
  - The U.S. Treasury yield curve (1M–30Y)
  - OECD 10-year government bond yields (country list)
- Displays two tabs:
  a) **Time Series**: yield evolution over time.
  b) **Latest Snapshot**: current yields in a sorted bar chart.
- Allows:
  - Start date selection.
  - Series toggles (U.S. curve and/or countries).
- Includes data export via `st.download_button`.

3. Data Sources & Series

FRED via pandas_datareader

U.S. Treasuries (daily, percent)

`DGS1MO, DGS2MO, DGS3MO, DGS4MO, DGS6MO, DGS1, DGS2, DGS3, DGS5, DGS7, DGS10, DGS20, DGS30`

OECD 10-Year Government Bond Yields (monthly, percent)

`IRLTLT01USM156N, IRLTLT01DEM156N, IRLTLT01FRM156N, IRLTLT01ITM156N, IRLTLT01GBM156N, IRLTLT01JPM156N, IRLTLT01ESM156N, IRLTLT01PTM156N, IRLTLT01GRM156N`

4. Environment & Setup

Install required packages:

```
pip install pandas pandas_datareader matplotlib numpy streamlit
```

Only one file is required: `app.py`.

5. Series Definitions & Naming

```
us_yields = [
    "DGS1MO", "DGS2MO", "DGS3MO", "DGS4MO", "DGS6MO",
    "DGS1", "DGS2", "DGS3", "DGS5", "DGS7", "DGS10", "DGS20", "DGS30"
]

yields_10y = [
    "IRLTLT01USM156N", "IRLTLT01DEM156N", "IRLTLT01FRM156N",
    "IRLTLT01ITM156N", "IRLTLT01GBM156N", "IRLTLT01JPM156N",
    "IRLTLT01ESM156N", "IRLTLT01PTM156N", "IRLTLT01GRM156N"
]

names = {
    "CPIAUCNS": "CPI",
    "GDP": "GDP",
    "DGS10": "U.S. 10Y Treasury",
    "IRLTLT01DEM156N": "Germany",
    "IRLTLT01FRM156N": "France",
    "IRLTLT01ITM156N": "Italy",
    "IRLTLT01GBM156N": "United Kingdom",
    "IRLTLT01JPM156N": "Japan",
    "IRLTLT01ESM156N": "Spain",
    "IRLTLT01PTM156N": "Portugal",
    "IRLTLT01GRM156N": "Greece",
}
```

6. Data Download & Caching

```python
import pandas as pd
import pandas_datareader.data as web
import streamlit as st

@st.cache_data
def download_fred_series(series_names, start_date="1990-01-01"):
    """Fetch FRED time series and return a wide DataFrame."""
    df = web.DataReader(series_names, "fred", start=start_date)
    return df.ffill()
```

7. Visualization Helpers (Matplotlib)

Time Series Lines

```python
import matplotlib.pyplot as plt

def plot_timeseries_lines(dataframe, title, y_label=""):
    fig, ax = plt.subplots(figsize=(10, 5))
    dataframe.plot(ax=ax)
    ax.set_title(title)
    ax.set_xlabel("Date")
    ax.set_ylabel(y_label)
    ax.legend(fontsize="small", ncol=2)
    fig.tight_layout()
    return fig
```

Latest Snapshot Bar Chart

```python
import numpy as np

def plot_bar_snapshot(dataframe, title, y_label=""):
    latest = dataframe.ffill().iloc[-1].sort_values()
    y_min = np.floor(latest.min())
    y_max = np.ceil(latest.max()) + 1

    fig, ax = plt.subplots(figsize=(8, 5))
    latest.plot(kind="bar", ax=ax, color="skyblue", edgecolor="black")
    ax.set_title(title)
    ax.set_ylabel(y_label)
    ax.set_ylim(y_min, y_max)
    ax.set_xticklabels(latest.index, rotation=0, ha="center")

    for i, v in enumerate(latest):
        ax.text(i, v + 0.05, f"{v:.2f}", ha="center", va="bottom", fontsize=9)

    fig.tight_layout()
    return fig
```

8. Streamlit App Structure

Students must create a **Streamlit dashboard** that separates the U.S. yield curve and the OECD 10-year yields, both as current snapshots and as historical time series. The layout must include **four visualizations** and an option to export data.

8.1 Sidebar Controls

- Include a date input widget to select the start date for the analysis.
- Display the selected start date on the sidebar.
- All data downloads must be cached using `@st.cache_data` to avoid redundant requests.

8.2 Main Layout

The main page should be divided into two sections: *Latest Snapshots* and *Time-Series Evolution*.

A. Latest Snapshots

Display two **bar charts** side by side:

(1) **OECD 10-Year Government Bond Yields** — one bar per country, sorted from lowest to highest.
   - Add a caption with the latest available observation date.
   - The y-axis limits should be rounded to neat integer values for readability.
   - Annotate each bar with the corresponding yield value.

(2) **U.S. Yield Curve Snapshot** — most recent yields across maturities (1M–30Y).
   - Bars should appear in order of maturity from left to right.
   - Annotate each bar with its value in percentage points.
   - The chart should show the current shape of the yield curve (normal, flat, or inverted).

B. Time-Series Evolution

Below the snapshots, include two **line charts** placed side by side:

(1) **OECD 10-Year Yields Over Time** — one line per country, spanning the selected date range.
   - This chart illustrates how long-term sovereign yields move relative to each other.
   - Use consistent colors and clear legends for each country.

(2) **U.S. Treasury Yields Over Time** — one line per maturity (1M–30Y).
   - Highlights changes and potential inversions in the yield curve.
   - Axes and legends should match the styling of the OECD chart for visual coherence.

8.3 Export Functionality

- Combine the two datasets (U.S. yields and OECD 10-year yields) into a single table aligned by date.
- Add a **"Download CSV"** button that exports the combined dataset as a CSV file.
- Ensure the exported file includes readable column names derived from the `names` dictionary.

8.5 Yield Curve Surface Visualization

Students can extend their dashboard by adding a **3D surface plot** that visualizes the evolution of the U.S. yield curve over time. This plot provides an intuitive, continuous representation of how yields at different maturities change across days.

Purpose

The goal is to display the yield curve as a dynamic surface:

- The **x-axis** represents the yield maturity (e.g., 1M, 3M, 6M, 1Y, 2Y, 5Y, 10Y, 30Y).
- The **y-axis** represents the time dimension (calendar date or trading day).
- The **z-axis** shows the yield level (in percent).

This allows the viewer to see both cross-sectional (by maturity) and temporal (by date) variations of the yield curve.

Data Preparation

- Use the U.S. Treasury constant-maturity series (`DGS1MO` through `DGS30`).
- Clean and align all maturities to a common daily index using forward-fill interpolation.
- Optionally, restrict the visualization to the last 1–2 years to improve rendering performance.

Visualization Requirements

- The plot should appear in a separate section or tab titled **"Yield Curve Surface"**.
- Use a 3D surface or wireframe style plot (e.g., with Matplotlib's `Axes3D`).
- Label axes clearly:
  - X-axis: "Maturity (Years)"
  - Y-axis: "Date"
  - Z-axis: "Yield (%)"
- Apply a color map (e.g., `viridis` or `plasma`) to emphasize yield variations.
- Include a color bar legend to interpret yield levels visually.

Interpretation Guidelines

Students should briefly discuss:

- How the yield curve steepens or flattens across time.
- Any observable periods of inversion or rapid shifts in curve shape.
- The impact of monetary policy changes on curve dynamics.

Performance Considerations

- Cache the prepared 3D surface data to avoid recomputation.
- Limit the number of plotted dates if rendering becomes slow.
- Ensure the color scale and axis limits adjust automatically to data ranges.

This surface visualization provides a more advanced extension of the project, helping users view the yield curve as a continuous, time-evolving object rather than discrete lines.